Hypergraphes acycliques et résolution de contraintes

Nathan TILLIER

13 octobre 2025

Table des matières

1 Généralités sur les hypergraphes **3**
1.1 Quelques définitions 3
1.2 Développements sur l'acyclicité 4

2 Algèbre relationnelle **6**
2.1 Relations, schémas et instances 6
2.2 Requêtes conjonctives 7

3 Application aux bases de données **11**
3.1 Acyclicité et arbres de jointure 11
3.2 Réduction Graham-Yu-Ozsoyoglu 12
3.3 Algorithme de Yannakakis 13

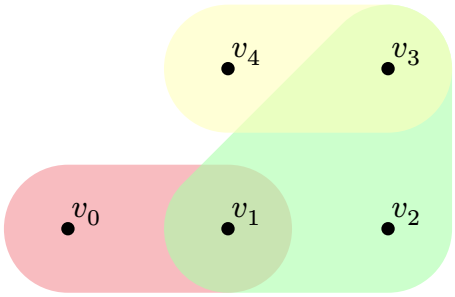
4 Tétra **14**
4.1 Chronologie 14
4.2 Résultats expérimentaux 14
4.3 Conclusion 14

Bibliographie **15**

1 Généralités sur les hypergraphes

La théorie des hypergraphes est une généralisation de la théorie des graphes, en particulier ceux non orientés, dans laquelle une arête peut relier plus de deux sommets entre eux. Les dessins ont l'avantage d'être proches, avec les arêtes (« hyperarêtes ») représentées par des tracés fermés entourant les sommets.

La figure ci-dessous en donne un aperçu.



Sur cet hypergraphe, une première arête rouge relie les sommets v_0 et v_1, une seconde jaune les sommets v_3 et v_4, tandis qu'une dernière (verte) relie les sommets v_1, v_2 et v_3.

1.1 Quelques définitions

Dans le cadre de la théorie des hypergraphes, on appelle les arêtes des hyperarêtes. On fera souvent (voire systématiquement) un abus de language en les appelant arête comme pour un graphe.

Définition 1. (Hypergraphe) Soit $X = \{x_1, x_2, ..., x_n\}$ un ensemble fini. Un hypergraphe sur X est une famille $H = (E_1, E_2, ..., E_p)$ de parties de X vérifiant :

(i) $$\forall i \in [\![1, p]\!],\ E_i \neq \emptyset$$

(ii) $$\bigcup_{i=1}^{p} E_i = X$$

Quelques observations :

- les éléments de X sont les sommets de l'hypergraphe, et les éléments de la famille H sont les arêtes de l'hypergraphe. On réduira souvent l'hypergraphe à H la famille de ses arêtes, X étant défini implicitement par l'union des éléments de H.
- une autre notation présente dans la littérature est $H = (V, \mathcal{E})$ où V est l'ensemble des sommets et $\mathcal{E}$ est la famille des hyperarêtes.
- un graphe est alors un hypergraphe dont toutes les arêtes relient deux sommets *exactement* (nécessairement différents si on n'autorise pas la présence de boucles)

$$\forall i, |E_i| = 2$$

Remarque. Cette définition[1] est assez contraignante. On peut en trouver une autre (sur *Wikipédia* notamment) ne mentionnant pas la seconde condition. D'autres propriétés figurant dans [2] peuvent faire office d'introduction aux hypergraphes (et aux matroïdes).

Un hypergraphe n'a, avec la définition précédente, aucun sommet isolé.

1.2 Développements sur l'acyclicité

La généralisation de l'acyclicité aux hypergraphes est ambigüe : réaliser un cycle revient essentiellement à partir d'un sommet puis d'y retourner sans « revenir sur son chemin ». Comme toute généralisation qui se respecte, la restriction de la condition d'acyclicité doit rester valide dans le cadre d'un graphe non orienté.

Remarque. Les définitions suivantes proviennent de la thèse de David Duris [3], soutenue en 2009 (donc relativement ancienne).

Plusieurs définition de l'acyclicité ont donc vu le jour : l'α-acyclicité, la β-acyclicité, la γ-acyclicité et la Berge-acyclicité.

Ces différences proviennent du point de vue porté sur les hypergraphes :

- la Berge-acyclicité nous vient de Claude Berge.
- la β-acyclicité de la combinatoire (et de Berge aussi). Un hypergraphe β-acyclique (*super-balanced hypergraph* en anglais), partage certaines caractéristiques des graphes bipartis[2].
- l'α-acyclicité et la γ-acyclicité tirent les origine des bases données. On s'y intéressera donc tout particulièrement.

Remarque. Il est important de noter que ces définitions de sont pas équivalentes ! Elles traduisent des propriétés certes similaires mais sensiblement différentes.

Dans certaines définitions d'acyclicité qui seront données, un chemin sera décrit pas une suite alternant entre arête et sommet. Les arêtes constituent le « véhicule » permettant de passer d'un sommet à l'autre, et deux sommets peuvent avoir plusieurs arêtes communes par lesquelles on peut passer.

Par convention, un cycle commencera par une arête et terminera par un sommet appartenant à cette première arête.

[1] Donnée notamment par Claude Berge, ancien directeur de recherches au CNRS, dans son ouvrage *Graphes et hypergraphes* [1]

[2] https://en.wikipedia.org/wiki/Balanced_hypergraph

Définition 2. (Berge-acyclicité) Soit H un hypergraphe sur X.

Un *Berge-cycle* est une suite $(E_1, x_1, ..., E_p, x_p)$ où $p \geq 2$ telle que

(i) les E_i sont des arêtes distinctes

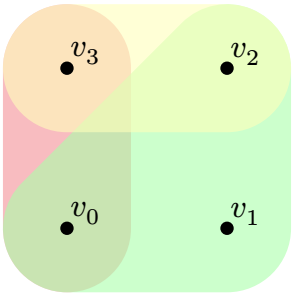
(ii) les x_i sont des sommets distincts

(iii) pour tout $i \in [\![1, p]\!]$, $x_i \in E_i \cap E_{i+1}$ (avec $E_{n+1} = E_1$)

H est dit *Berge-acyclique* s'il ne contient aucun *Berge-cycle*.

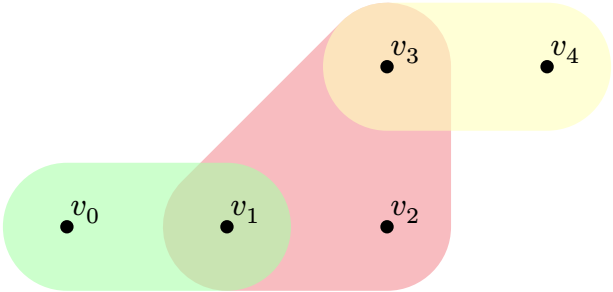
Dans l'essentiel, cela revient à dire qu'on a trouvé un chemin partant et revenant à x_1 par des sommets et arêtes distinctes.

Attention ! L'une de ces conditions n'implique pas l'autre, contrairement aux chemins élémentaires et simples dans des graphes, où un chemin élémentaire est forcément simple.



Ici, passer par les sommets (v_0, v_1, v_2, v_3) distincts ne garantit pas que les arêtes soient distinctes (parce qu'une arête n'est pas définie par un couple de deux sommets).

L'hypergraphe suivant est Berge-acyclique en revanche.



La Berge-acyclicité est sans doute la généralisation la plus directe de l'acyclicité aux hypergraphes. Ce sera en revanche l'α-acyclicité qui nous intéressera le plus (voir l'application aux bases de données)

2 Algèbre relationnelle

Tout d'abord, quelques rappels sur l'algèbre relationnelle, qui est une formalisation mathématique des opérations réalisables sur une base de données.

2.1 Relations, schémas et instances

Le schéma d'une base de données est un ensemble de relations R, S, T. Les relations sont en quelques sorte les briques élémentaires des bases de données.

Définition 3. (Relation) Une relation est un objet suivant un schéma de la forme

$$\text{rel}(\text{a1} : \text{dom1}, ..., \text{an} : \text{domn})$$

On appelle
- rel nom de la relation
- a1, ..., an les noms des attributs
- dom1, ..., domn les domaines des attributs

Par exemple, ce qui suit peut définir une relation

$$\text{commune}(\text{code} : \text{entier}, \text{département} : \text{entier}, \text{nom} : \text{texte})$$

On appelle arité d'une relation le nombre d'attributs (n dans la définition ci-dessus). On remarque au passage qu'il y a deux manière de voir les couples attribut × domaine : sous la forme d'un n-uplet où les valeurs sont numérotées, ou bien sous la forme d'un dictionnaire, où chaque attribut est accessible à partir de son nom.

On note $\mathbf{dom}(A)$ le domaine de A.

Définition 4. (Instance) Un *n-uplet* sur une relation R est une valeur possible d'un élément de la relation, de la forme $R(a_1, ..., a_n)$ (une occurence de R).

Une instance I sur la relation R est un ensemble fini de n-uplets sur R, qu'on note R^I.

Une instance I sur le schéma d'une base de données est l'union des instances des relations R du schéma.

Par exemple, si on prend les relations

$$\text{commune}(\text{nom} : \text{texte}, \text{département} : \text{entier})$$

$$\text{département}(\text{numéro} : \text{entier}, \text{nom} : \text{texte}, \text{région} : \text{texte})$$

Alors un schéma de base de données $\boldsymbol{R}$ possible est $\{\text{commune}, \text{département}\}$.

Une instance I sur $\boldsymbol{R}$ sera donc un ensemble de relations commune ou département :

$$I = \{\text{commune}(\text{Brest}, 29), \text{commune}(\text{Paris}, 75),$$
$$\text{département}(29, \text{Finistère}, \text{Bretagne}), \text{département}\big(75, \text{Paris}, \text{Île-de-France}\big)\}$$

On rappelle brièvement ce que sont une projection et une sélection.

Définition 5. (Projection) Si R est une relation et $H = (A_1, ..., A_n)$ des attributs de R, on appelle projection de R sur H et on note $\pi_H R$ la relaiton obtenue à partir de R en ne gardant que les attributs $A_1, ..., A_n$ et en supprimant les doublons.

Quand on réalise une projection, on perd (temporairement) de l'information en enlevant des attributs de la relation, ce qui est parfois utile quand certaines informations ne nous intéressent pas.

Définition 6. (Sélection) Soient R une relation et E une expression logique portant sur les attributs de R. On appelle sélection de R selon E (qu'on note $\sigma_E R$) la relation obtenue à partir de R en ne gardant que les n-uplets de R satisfaisant l'expression logique E.

Dans la suite, on ne s'autorisera que des expressions logiques avec des égalités, l'algorithme décrit fonctionnant dans ce cadre précis.

2.2 Requêtes conjonctives

Les requêtes conjonctives permettent de formaliser la recherche de n-uplets vérifiant certaines relations sur un schéma de données $\boldsymbol{R}$.

Définition 7. (Requête conjonctive) On note les requêtes conjonctives sous la forme

$$q(x_1, ..., x_n) : -R_1(\vec{y_1}), ..., R_p\big(\vec{y_p}\big)$$

La première partie désigne $q(x_1, ..., x_n)$ l'en-tête de la requête, et la seconde $R_1(\vec{y_1}), ..., R_p\big(\vec{y_p}\big)$ est le corps cette même requête. On appelle $\mathbf{var}(q)$ l'ensemble des variables de la requête, et chacune des expressions $R_i(\vec{y_i})$ est appelé clause atomique ou atome.

Cette expression doit toutefois vérifier quelques conditions :

- chaque n-uplet $\vec{y_i}$ doit avoir la même taille que l'arité de la relation correspondante (pour que ce soit bien définit)
- chaque variable de l'en-tête doit être présente dans un moins l'un des $\vec{y_i}$.

À première vue cette expression est rébarbative, mais elle est en fait assez logique (ouf !) :

- la notation $\vec{y_i}$ reprend l'idée qu'un élément sur une relation peut être vu comme un n-uplets, autrement dit comme un vecteur de la base de données.

- les variables x_i constituant le vecteur $\vec{x}$ de l'en-tête représentent les champs qu'on va sélectionner. C'est la projection de la sélection opérée sur $\boldsymbol{R}$ à partir du corps de la requête.
- le corps de la requête quant à lui permet de quantifier la requête effectuée sur l'instance I de la base de données.

Reprenons l'instance I définie ci-dessus comme exemple.

$$I = \{\text{commune}(\text{Brest}, 29), \text{commune}(\text{Paris}, 75),$$
$$\text{département}(29, \text{Finistère}, \text{Bretagne}), \text{département}\big(75, \text{Paris}, \text{Île-de-France}\big)\}$$

À partir des relations

$$\text{commune}(\text{nom} : \text{texte}, \text{département} : \text{entier})$$

$$\text{département}(\text{numéro} : \text{entier}, \text{nom} : \text{texte}, \text{région} : \text{texte})$$

On considère la requête (conjonctive) suivante

$$q(x, y, z) :- \text{commune}(x, u), \text{département}(u, y, z)$$

Cette requête paraît obscure, mais il ne faut pas oublier que ce sont des variables muettes ! On peut donc la réécrire sous la forme

$$q(c, d, r) :- \text{commune}(c, u), \text{département}(u, d, r)$$

Ou (ce qui est encore plus clair)

$$q(\text{com}, \text{dep}, \text{reg}) :- \text{commune}(\text{com}, \text{id}), \text{département}(\text{id}, \text{dep}, \text{reg})$$

On est en fait en train de récupérer les noms des villes de l'instance I, associés à leur nom de département et nom de région ! On vérifie d'ailleurs au passage que cette vérifie bien les criètères donnés précédemment :

- les n-uplets ont la bonne taille et la bon typage
- chaque variable est présente dans au moins un des vecteurs donnés en entrée aux relations.

Il n'est pas trop difficile de trouver de tête le résultat de la requête, on introduit tout de même la notion de valuation pour expliciter la procédé par lequel on souhaite obtenir toutes les n-uplets vérifiant la requête.

Définition 8. (Valuation) Une valuation v sur un ensemble de variables V est une application de V dans le domaine **dom** de $\boldsymbol{R}$.

(Dans la pratique, il faudra que le typage soit correct, bien que cela ne pose pas problème du point de vue théorique[3]).

Dans la continuité de l'exemple précédent, une valuation possible est

[3] Je crois ?

$$v(x) = \begin{cases} \text{"Brest" si } \ x = \text{com} \\ \text{"Finistère" si } \ x = \text{dep} \\ \text{"Bretagne" si } \ x = \text{reg} \\ 29 \ \text{ si } \ x = \text{id} \end{cases}$$

On étend la notation de manière intuitive aux tuples :

$$v((x, y)) = (v(x), v(y))$$

Et aux n-uplets de manière générale

$$v((x_1, ..., x_n)) = (v(x_1), ..., v(x_n))$$

On est alors en mesure de définir le résultat d'une requête conjonctive sur une instance I :

$$q(I) = \{v(\vec{x}) \mid v \ \text{ valuation sur } \ \mathbf{var}(q) \ \text{ vérifiant pour tout } \ i, v(\vec{y_i}) \in R_i^I\}$$

De manière générale, toutes les requêtes contenant des sélections avec égalité, des projections et des jointures sont des requêtes conjonctives : c'est donc un des concepts fondamentaux des recherches de résultats dans des bases de données !

Quelques définitions complémentaires, qui pourront s'avérer utiles.

Définition 9. (Monotonie) Une requête q est monotones si n'importe quelles instances $I \subseteq J$, $q(I) \subseteq q(J)$.

En bref, rajouter des n-uplets dans l'instance ne diminuera jamais le nombre de résultats renvoyés par la requête.

Proposition 1. Toute requête conjonctive est monotone.

Preuve. Soit $t \in q(I)$.

Il existe alors une valuation v et un n-uplet x de val(R) telle que $t = v(\vec{x})$ et pour tout R_i relation du schéma, $v(\vec{y_i}) \in R_i^I$.

Comme $I \subseteq J$, $v(\vec{y_i}) \in R_i^J$ et donc $t \in q(J)$. □

Proposition 2. Toute requête conjonctive est satisfiable.

Remarque. Il est au passage intéressant de noter que :

- savoir si une requête conjonctive est inclue dans un autre
- savoir si deux requêtes conjonctives sont équivalentes
- évaluer une requête conjonctive
- simplifier une requête conjonctive

…sont des problèmes NP-complets.[4]

Ce qui justifie le fait de s'intéresser au cas particulier des requêtes acycliques !

[4] D'ailleurs, certains autre problèmes liés sont non décidables dans toutes les algèbres relationnelles, et le premier point, si on s'autorise des comparaison différentes de l'égalité ($>$ ou $\neq$) sont bien plus que NP-complet (leur classe de complexité est Π_2^p)

3 Application aux bases de données

Remarque. La majorité des éléments de cette partie (algorithmes et autres) tirent leur origine de ce papier [4] (anglophone) sur l'optimisation des recherches acycliques ainsi que de [5]

3.1 Acyclicité et arbres de jointure

Dans le cadre de l'application aux bases de données la notion d'arbre de jointure est particulièrement intéressante. Tout hypergraphe n'admet pas nécessairement un arbre de jointure, et il est défini de la manière suivante sous réserve d'existence.

On s'intéresse tout d'abord à la manière dont on va contruire l'hypergraphe, à partir d'une requête conjonctive.

Définition 10. (Hypergraphe d'une requête) Soit une requête acyclique q.

$$q(x_1, ..., x_n) : -R_1(\vec{y_1}), ..., R_p(\vec{y_p})$$

On pose l'hypergraphe $H(q) = (V, E)$ associé à q de la façon suivante :

- l'ensemble des sommets V contient toutes les variables du corps de q
- l'ensemble des arêtes E contient, pour chaque clause atomique de la requête, l'ensemble des variables apparaissant dedans.

On cherche ensuite un cas dans lequel la requête est plus simple à utiliser (*i.e.* avec moins de conditions contraignantes). Une manière est de cherche un arbre de jointure sur cet hypergraphe.

Définition 11. (Arbre de jointure) Soit H un hypergraphe sur X. On considère T un graphe ayant pour sommets les arêtes de H.

T est un arbre de jointure de H si et seulement si

(i) Pour tout sommet v de H, le sous-graphe de T engendré par l'ensemble des sommets de T contenant v est connexe.

(ii) T est un arbre.

Bien évidemment, un des buts sera de déterminer s'il existe un arbre de jointure (réduction GYO). Cet arbre de jointure n'est bien évidemment pas unique.

Définition 12. (α-acyclicité) Soit H un hypergraphe sur X. H est α-acyclique s'il contient un arbre de jointure.

Cette définition d'α-acyclicité ne va pas de soi…mais ce n'est pas là où il faut mettre l'accent. En revanche, un point qui peut poser problème est la connexité : ce qui nous intéresse est que la requête soit acyclique, connexe ou non.

On utilise alors, de manière similaire à la théorie des graphes, le vocable de forêt.

Définition 13. (Forêt de jointure) Un forêtde jointure d'une requête conjonctive est une forêt $F = (V, E)$ dont tous les sommets sont des clauses atomiques de q et tel que pour chaque paire de clauses atomiques R et S ayant des variables en commun, les condition suivantes soient vérifiées :

- R et S appartiennent à la même composante connexe.
- toutes les variables communes à R et S sont présentes sur un unique chemin de R à S.

On appelle F arbre de jointure si c'est un arbre.

Proposition 3. Une requête conjonctive est acyclique si et seulement si elle possède une forêt de jointure.

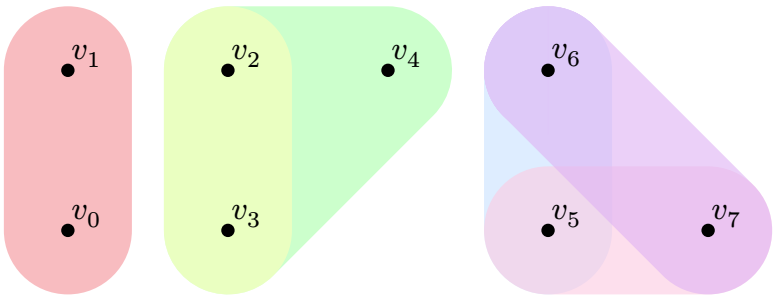
3.2 Réduction Graham-Yu-Ozsoyoglu

Les algorithmes décris dans cette partie proviennent en partie de [5]. Il est intéressant de remarquer que l'approche diffère légèrement de [3] et [6] par la définition du procédé. L'algorithme présenté est plus pratique dans le cadre théorique, l'implémentation différera légèrement pour des raisons de commodité.

Pour cela on définit *an hear*, innocemment traduit en français par oreille[5].

Définition 14. (Oreille) Une oreille d'un hypergraphe H est une arête E telle qu'il existe une autre arête F appelée *arête témoin* vérifiant

Pour tout sommet $x \in E$ non exclusivement dans E, $x \in F$.



Dans l'hypergraphe ci-dessus, seules les arêtes rouges, vertes et jaunes sont des oreilles.

[5] À vrai dire, je n'ai pas cherché la traduction académique française.

L'algorithme suivant, dit de réduction GYO (pour Graham-Yu-Ozsoyoglu) permet de détecter l'α-acyclicité d'un hypergraphe.

Graham-Yu-Ozsoyoglu

Entrée : hypergraphe H
Sortie : booléen indiquant si H est acyclique

Tant que H a une oreille **faire**
  $e \leftarrow$ oreille de H
  retirer toutes les arêtes exclusivement dans e
  retirer e
Fin

Renvoyer H est l'hypergraphe vide

Algorithme 1. – Graham-Yu-Ozsoyoglu

L'implémentation consiste par contre à appliquer successivement ces deux règles :
- on enlève tous les sommets présents dans une seule arête
- si une arête est incluse dans une autre arête, on la retire

On peut montrer que l'hypergraphe est α-acyclique si et seulement si on obtient l'hypergraphe vide à la fin de cette procédure.

L'idée est la suivante :
- dès qu'on retire un sommet isolé, on ne perd aucune information sur T. On retire d'ailleurs son arête avec si elle ne contient que lui.
- si une arête E est incluse dans une autre arête F, alors les sommets E et F sont nécessairement reliés dans T (on crée la connexité sans briser l'acyclicité).
- si une de ces étapes n'est pas faisable et que l'hypergraphe n'est pas vide, on ne peut pas contruire T : on créera forcément un cycle.

Rappel. Faire la complexité théorique, avec l'implémentation sous-jacente.

3.3 Algorithme de Yannakakis

Cet algorithme est expliqué dans [5], la thèse [3] cite également le papier original et l'auteur (Yannakakis) à de nombreuses reprises. Il a été initialement présenté dans [7].

À faire :
- présenter l'algorithme, expliquer pourquoi il fonctionne (préparer des éléments de preuve)
- simuler « à la main » l'exécution d'une requête ?

4 Tétra

4.1 Chronologie

Présentation de quinze minutes : c'est très court. Il faudra donc s'en tenir à l'essentiel concernant les aspects théoriques. En revanche, bien mettre l'emphase sur les expérimentations avec les bases de données des communes, départements et régions !

Dans l'idéal, il faudra suivre cette chronologie:

- Introduction (2 minutes)
  - Graphe et acyclicité
  - Hypergraphes
  - Fil rouge : la base de données
- Aspect théorique (4 minutes)
  - Algèbre relationnelle
  - Requête conjonctive
  - Arbre de jointure
- Explication des algorithmes (4 minutes)
  - Réduction GYO
  - Algorithme de Yannakakis
- Résultats expérimentaux (3 minutes)
  - Implémentation finale
  - Implémentation naïve
  - Comparaison des performances
- Conclusion (2 minutes)

Le but va être de présenter l'implémentation de manière linéaire, en expliquant simultanément pourquoi ça fontionne, et comment je le fais fonctionner.

C'est *très* court.

4.2 Résultats expérimentaux

- comparer l'approche naïve (produits cartésiens et compagnie) avec celle de l'algorithme en fonction de la taille de la base de données et de certains paramètres
- l'exécuter sur les villes et départements de France

4.3 Conclusion

- insister sur le fait que c'est NP-difficile dans le cas général
- mettre les améliorations en perspective : existe-t-il des variantes plus efficaces dans certains cas particuliers différents ? Il y a-t-il des moyens d'utiliser cette approche pour des requêtes « presque » α-acyclique ?

Bibliographie

[1] C. Berge, *Graphes et hypergraphes*. in Dunod université: Mathématiques. Dunod, 1970. [En ligne]. Disponible sur: https://books.google.fr/books?id=OtFbJceS97EC

[2] V. Delaitre, « Hypergraphes et matroïdes ». [En ligne]. Disponible sur: https://perso.ens-lyon.fr/eric.thierry/Graphes2009/vincent-delaitre.pdf

[3] D. Duris, « Acyclicité des hypergraphes et liens avec la logique sur les structures relationnelles finies », 2009. [En ligne]. Disponible sur: http://www.theses.fr/2009PA077158

[4] Z. Luo, W. V. den Broeck, G. V. den Broeck, et Y. R. Wang, « Algorithms for Optimizing Acyclic Queries ». [En ligne]. Disponible sur: https://arxiv.org/abs/2509.14144

[5] P. Koutris, « Foundations of Data Management : Acyclic Conjunctive Queries ». [En ligne]. Disponible sur: https://pages.cs.wisc.edu/~paris/cs784-f19/lectures/lecture4.pdf

[6] J. Brault-Baron, « Hypergraph Acyclicity Revisited ». [En ligne]. Disponible sur: https://arxiv.org/abs/1403.7076

[7] M. Yannakakis, « Algorithms for Acyclic Database Schemes », in *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7*, 1981. [En ligne]. Disponible sur: https://www.researchgate.net/publication/200034379_Algorithms_for_Acyclic_Database_Schemes